vulkan-diagrams

Contents

- Introduction
- Boilerplate
- Render pass and swapchain
- Vertex buffer creation
- Vertex input and multiple bindings
- Descriptor sets
- Push constants
- Pipeline barriers
- Pipeline stages and access types
- Render loop
- Ray tracing

Introduction

Vulkan Diagrams is a collection of diagrams which are designed to serve as a quick reference for various topics in Vulkan. The diagrams show the Vulkan objects needed to accomplish common tasks (e.g. creating a vertex buffer) and the relationships between these objects.

For clarity, some members of Vulkan objects are omitted and some names are slightly simplified (e.g. changing `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL` to `SHADER_READ_ONLY_OPTIMAL`).

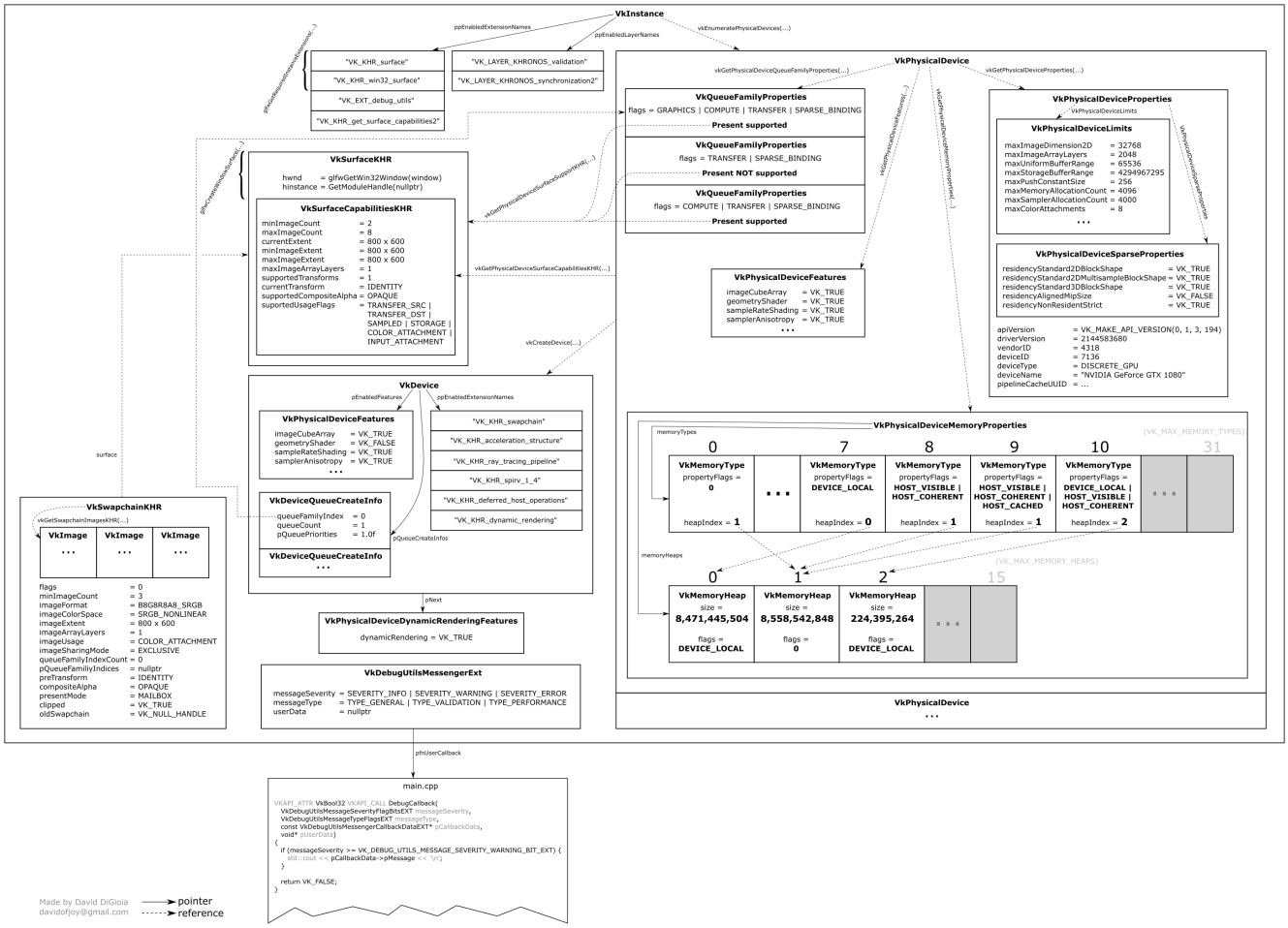
Boilerplate

This diagram uses example values from my GPU, an NVIDIA GeForce GTX 1080. Notice that memory heap 2 is both `DEVICE_LOCAL` and `HOST_VISIBLE`, which is good for GPU accessible memory that needs to be frequently updated from the CPU since it allows us to update it directly without using a staging buffer. On my machine it's only 224 MB, but on machines with resizeable bar (AMD's marketing term for this is Smart Access Memory) this heap will be significantly bigger.

Not shown in the diagram are the functions `vkEnumerateInstanceExtensionProperties(...)`, `vkEnumerateInstanceLayerProperties(...)`, and `vkEnumerateDeviceExtensionProperties(...)` to enumerate the available instance extensions, layers, and device extensions respectively.

Running vulkaninfo will print out specs about your GPU that are queriable in Vulkan. To find this info about other GPUs, gpuinfo is a good resource. If you want to temporarily override the validation layer settings you can use Vulkan Configurator.

Alternatively, vk-bootstrap is a good library that handles the Vulkan boilerplate.

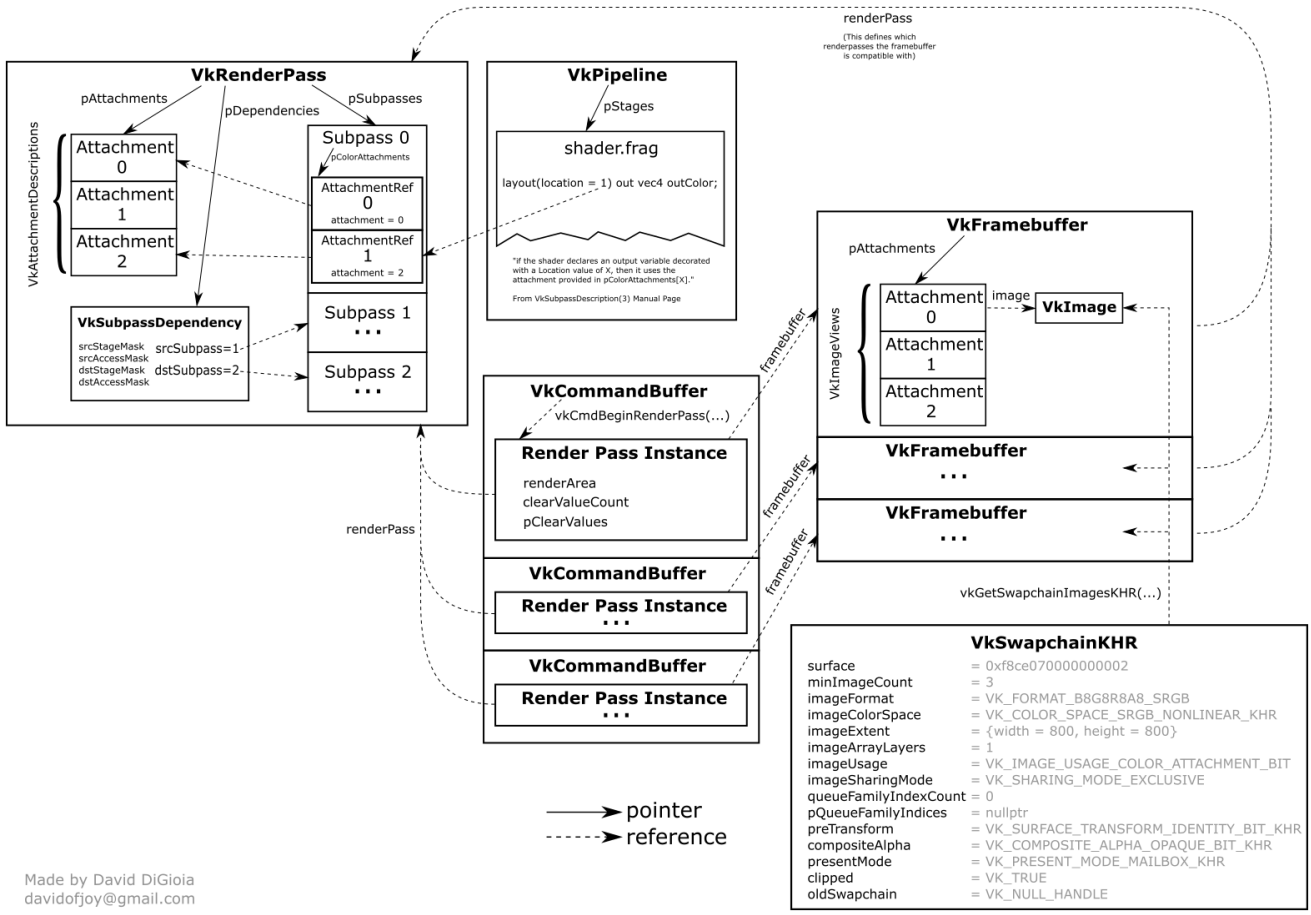


Render pass and swapchain

In this diagram, a single renderpass is used for each command buffer, and each renderpass has multiple subpasses.

You should use multiple subpasses instead of multiple render passes whenever possible. If a pass only needs to read from the one corresponding fragment in a previous pass, you can use a previous subpass as an input attachment and no additional render passes are needed. Here is an example of how to do that. If you need random access to a previous pass (to implement a guassian blur, for example) then it would be appropriate to use multiple render passes.

In the diagram we see that one of the attachments in the frame buffer has an image which is owned by the swapchain, but this is not mandatory. For example, you could render to a texture by creating your own `VkImage` with the usage flags `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT | VK_IMAGE_USAGE_SAMPLED_BIT` so it can be written into as a color attachment and then sampled from in a shader.



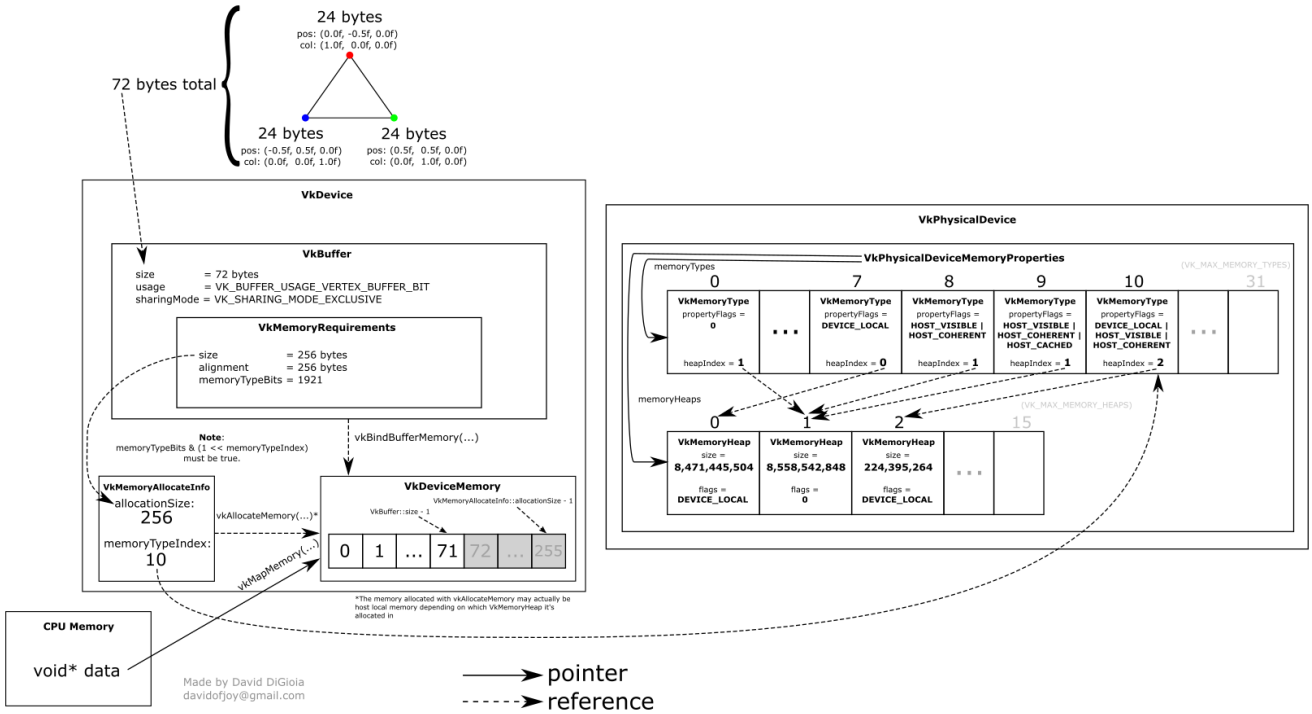
Vertex buffer creation

NVIDIA has a good article about memory management which I recommend reading. The takeaway is that it's usually preferable to make big memory allocations with big buffers and to

sub-allocate resources from those buffers. It includes the following descriptions of memory objects:

> **Heap** - Depending on the hardware and platform, the device will expose a fixed number of heaps, from which you can allocate certain amount of memory in total. Discrete GPUs with dedicated memory will be different to mobile or integrated solutions that share memory with the CPU. Heaps support different memory types which must be queried from the device.
> **Memory type** - When creating a resource such as a buffer, Vulkan will provide information about which memory types are compatible with the resource. Depending on additional usage flags, the developer must pick the right type, and based on the type, the appropriate heap.
>
> **Memory property flags** - These flags encode caching behavior and whether we can map the memory to the host (CPU), or if the GPU has fast access to the memory.
>
> **Memory** - This object represents an allocation from a certain heap with a user-defined size.
>
> **Resource (Buffer/Image)** - After querying for the memory requirements and picking a compatible allocation, the memory is associated with the resource at a certain offset. This offset must fulfill the provided alignment requirements. After this we can start using our resource for actual work.
>
> **Sub-Resource (Offsets/View)** - It is not required to use a resource only in its full extent, just like in OpenGL we can bind ranges (e.g. varying the starting offset of a vertex-buffer) or make use of views (e.g. individual slice and mipmap of a texture array).

Alternatively, Vulkan Memory Allocator (VMA) is a good library that handles memory allocations for you.



Vertex input and multiple bindings

`VkPipelineVertexInputStateCreateInfo` allows us to specify how our vertices are stored in memory. It is composed of an array of `VkVertexInputAttributeDescription`s and an array of `VkVertexInputBindingDescription`s.

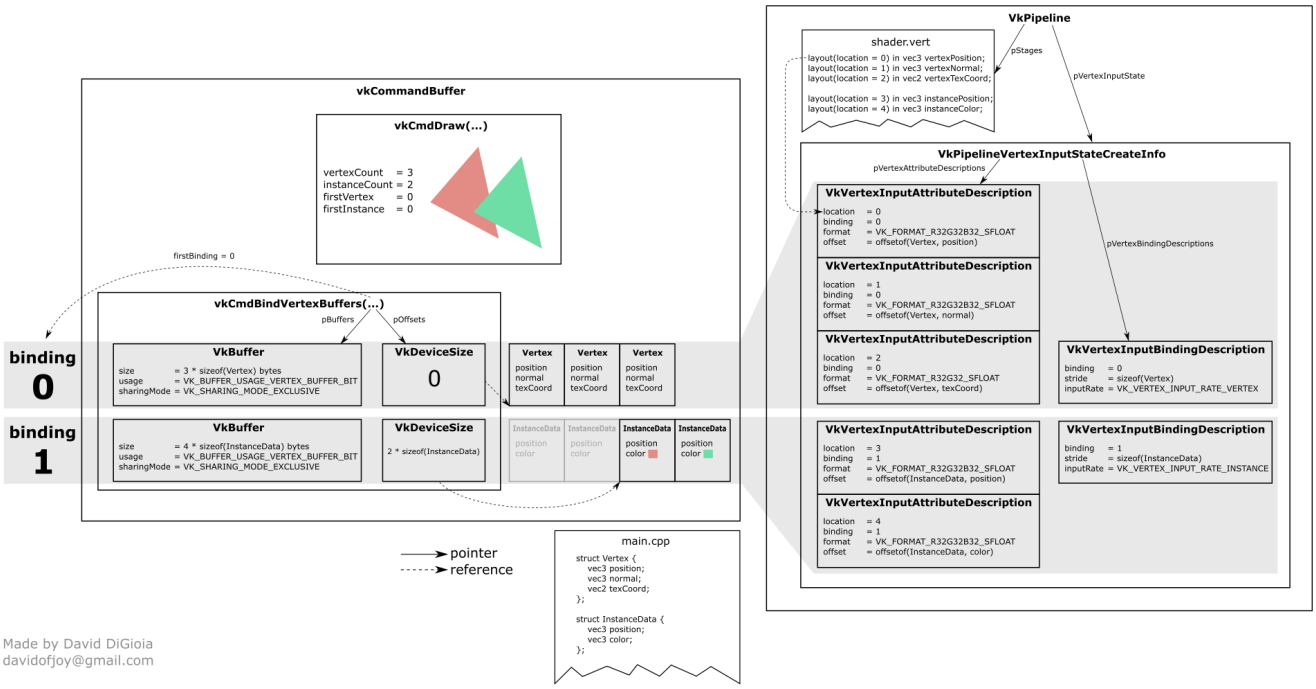
As the name implies, we will have one binding description for each binding. In this example, we see that binding 0 has `VK_VERTEX_INPUT_RATE_VERTEX` as its input rate, which means it increments to the next set of data `stride` apart for every *vertex*. Binding 1, on the other hand, has `VK_VERTEX_INPUT_RATE_INSTANCE` as its input rate, so we increment to the next set of data `stride` apart only for every *instance*. We specify the number of instances and vertices we draw in `vkCmdDraw`.

We have one vertex attribute for each member of the struct associated with that binding. For example, binding 0 has 3 vertex attributes since the vertex buffer bound to binding 0 is a buffer of `Vertex` structs which has members `position`, `normal` and `texCoord`. Binding 1 has only 2 vertex attributes since `InstanceData` has only 2 members. The format of each vertex attribute is determined by the size and type of that attribute, so some common choices include:

```
float:      VK_FORMAT_R32_SFLOAT
vec2:       VK_FORMAT_R32G32_SFLOAT
vec3:       VK_FORMAT_R32G32B32_SFLOAT
vec4:       VK_FORMAT_R32G32B32A32_SFLOAT
ivec2:      VK_FORMAT_R32G32_SINT
uvec4:      VK_FORMAT_R32G32B32A32_UINT
double:     VK_FORMAT_R64_SFLOAT
etc.
```

In this example we make one call to `vkCmdBindVertexBuffer` and pass in arrays to bind both buffers at the same time, but note it would have been possible to make two separate calls so long as in one call we specify `firstBinding = 0` and in the other `firstBinding = 1`.

Lastly, notice that the vertex shader is completely blind to which binding each of its input variables are coming from. The vertex shader only specifies the locations, then the bound buffer the data comes from for each variable is determined by the corresponding `VkVertexInputAttributeDescription`.



Descriptor sets

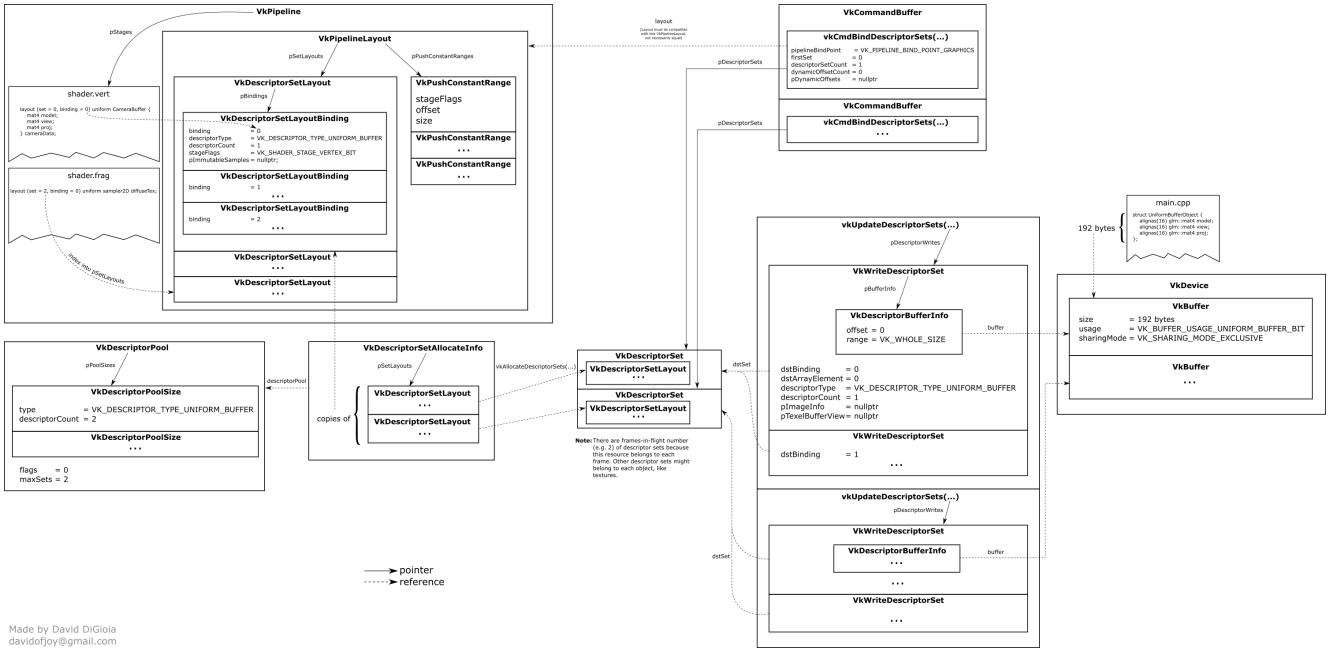
The best mental model description I've come across for some of the objects relating to descriptor

sets are from a comment in the Vulkan subreddit:

> **DescriptorPool** - A big heap of available UBOs, textures, storage buffers, etc that can be used when instantiating DescriptorSets. This allows you to allocate a big heap of types ahead of time so that later on you don't have to ask the gpu to do expensive allocations.
> **DescriptorSetLayout** - Defines the structure of a descriptor set, a template of sorts. Think of a `class` or `struct` in C or C++, it says "I am made out of, 3 UBOs, a texture sampler, etc". It's analogous to going
>
> ```
> struct MyDesc {
>   Buffer MyBuffer[3];
>   Texture MyTex;
> }
>
> struct MyOtherDesc {
>   Buffer MyBuffer;
> }
> ```
>
> **DescriptorSet** - An actual instance of a descriptor, as defined by a DescriptorSetLayout. Using the class/struct analogy, it's like going `MyDesc DescInstance();`
>
> **PipelineLayout** - If you treat your entire shader as if it was just one big `void shader(arguments)` function then a PipelineLayout is like describing all the "arguments" passed into your shader such as `void shader(MyDesc desc, MyOtherDesc otherDesc)` . This generally maps up to statements like `layout(std140,set=0, binding = 0) uniform UBufferInfo{Blah MyBlah;}` and `layout(set=0, binding = 2, rgba32f) uniform image2D MyImage;` in your shader code.
>
> **vkCmdBindDescriptorSet** - This is the mechanism to actually pass a DescriptorSet into a shader(aka pipeline). So basically passing the "arguments" like `shader(DescInstance,OtherDescInstance)` .

Note that `vkUpdateDescriptorSets(...)` doesn't copy a buffer into the descriptor set, but rather gives the descriptor set a pointer to the buffer described by `VkDescriptorBufferInfo`. So then `vkUpdateDescriptorSets(...)` doesn't need to be called more than once for a descriptor

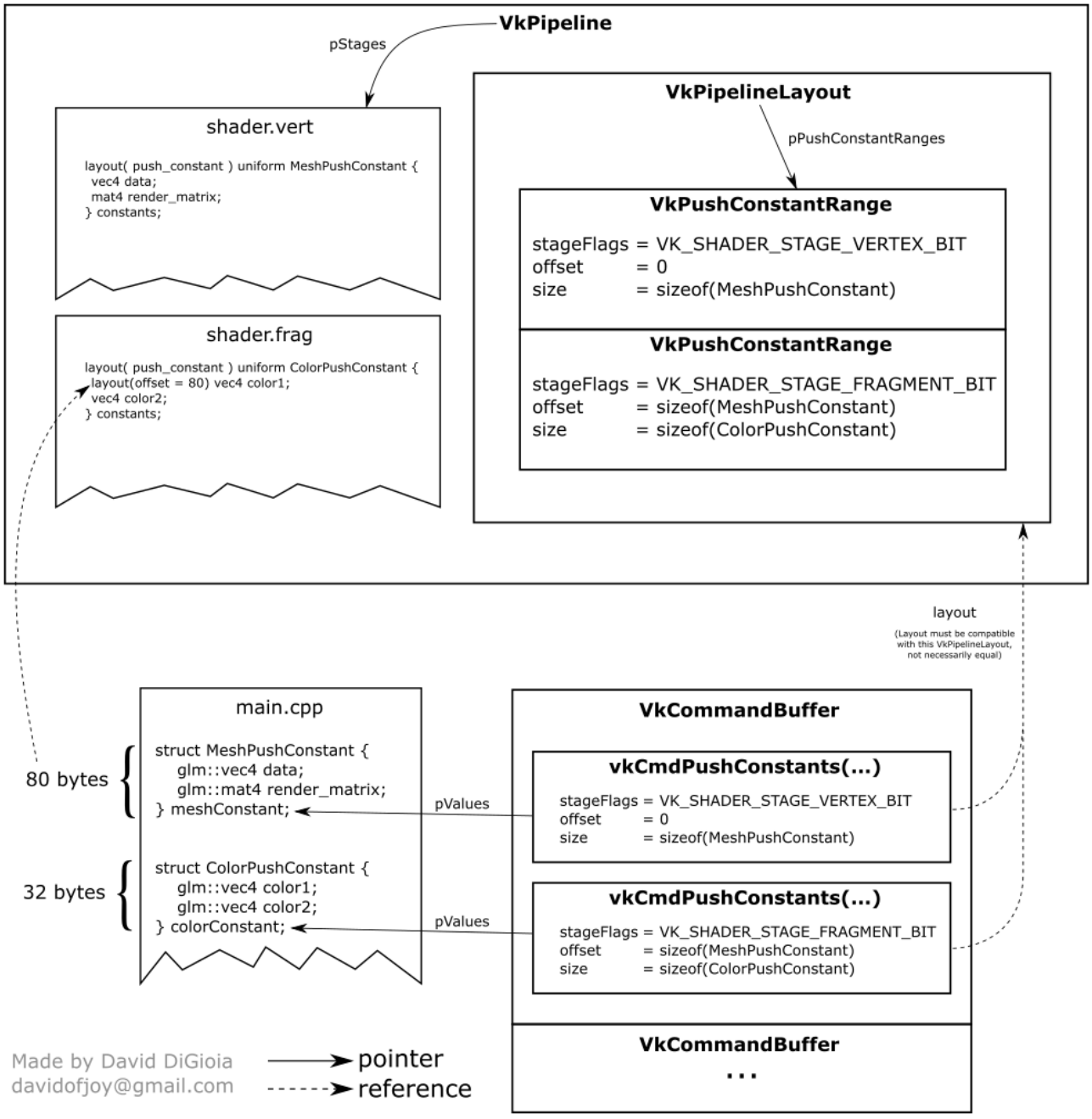
set since modifying the buffer that a descriptor set points to will update what the descriptor set sees.



Push constants

Push constants are a small amount of shader-accessible data that is written into the command buffer itself. The spec guarantees only 128 bytes of push constant space, though the exact limit can be found from `VkPhysicalDeviceLimits::maxPushConstantsSize`.

The diagram shows how to use ranges and offsets to make some one range available from the vertex shader, and the other from the fragment shader, but it's also possible to specifiy only one range which is available from both with `stageFlags` set equal to `VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT`;



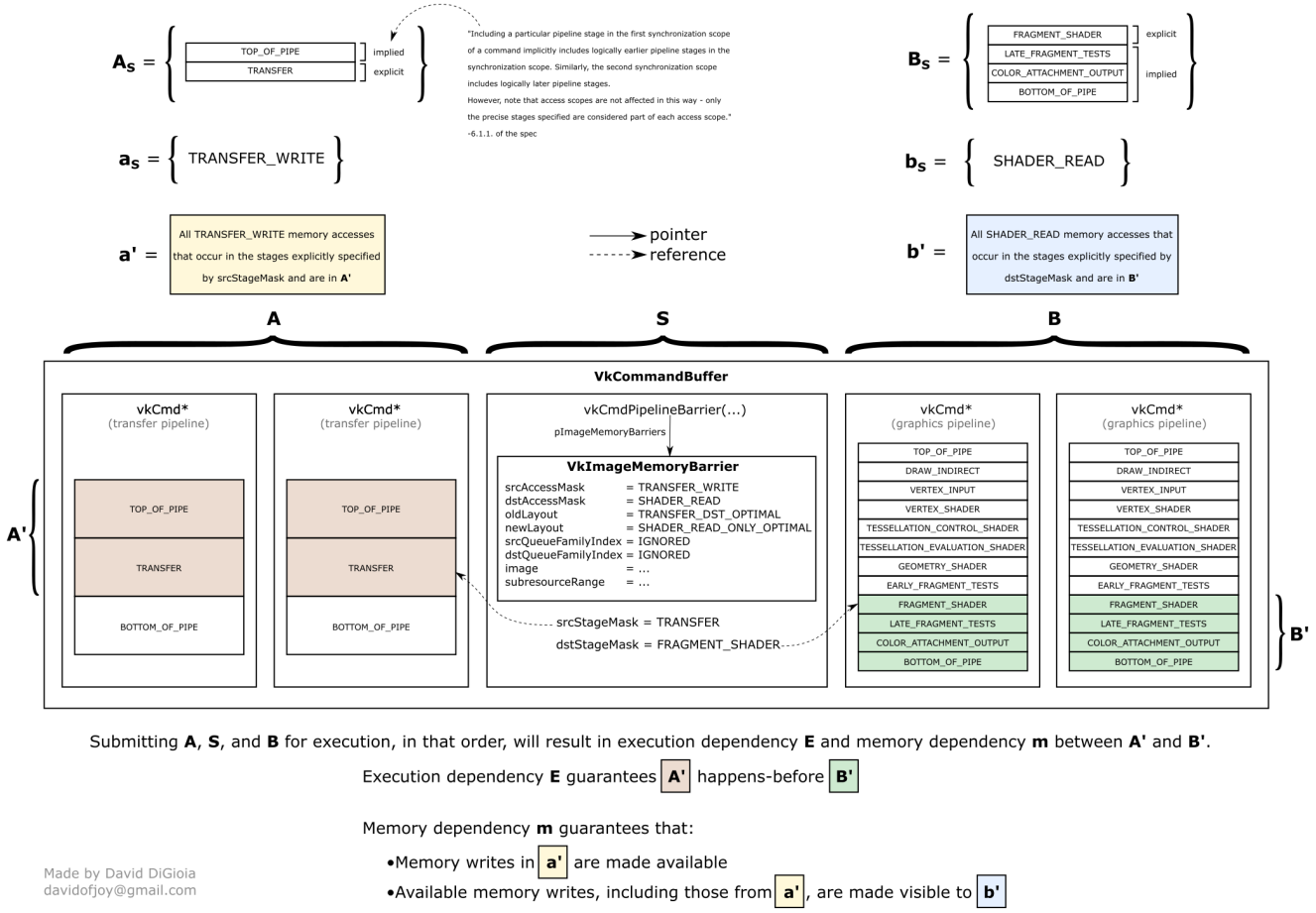
Pipeline barriers

This diagram shows the general use of pipeline barriers and how they create execution dependencies and memory dependencies. The specific example in the diagram shows the pipeline barrier which transfers the image's layout from `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL` to `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`. This is performed after copying the image data from a buffer into the image, to prepare the image

to be read from shaders. This example was taken from the texture mapping chapter of Vulkan Tutorial.

The set names and quotes are taken directly from the spec.

(ctrl+f search for PDF version of spec: ***Execution and Memory Dependencies***)



Submitting **A**, **S**, and **B** for execution, in that order, will result in execution dependency **E** and memory dependency **m** between **A'** and **B'**.

Execution dependency **E** guarantees **A'** happens-before **B'**

Memory dependency **m** guarantees that:

- Memory writes in **a'** are made available
- Available memory writes, including those from **a'**, are made visible to **b'**

Made by David DiGioia
davidofjoy@gmail.com

Pipeline stages and access types

When synchronizing using a pipeline barrier or subpass dependency, it's necessary to specify source and destination stages/access types. The following tables show each pipeline, all of their stages in the order they occur, and the access types supported by each of them. `VK_ACCESS_MEMORY_READ_BIT` and `VK_ACCESS_MEMORY_WRITE_BIT` are supported for all stages, but are not shown in the tables for clarity.

There are corresponding `VkPipelineStageFlagBits` and `VkAccessFlagBits` for each item in the Stage and Access Types columns respectively.

(ctrl+f search for PDF version of spec: ***Table 4. Supported access types*** and ***The graphics pipeline executes the following stages***)

Graphics pipeline:

Stage	Access Types
Draw Indirect	Indirect Command Read
Index Input	Index Read
Vertex Attribute Input	Vertex Attribute Read
Vertex Shader	Uniform Read Shader Read Shader Write Acceleration Structure Read
Tessellation Control Shader	Uniform Read Shader Read Shader Write Acceleration Structure Read
Tessellation Evaluation Shader	Uniform Read Shader Read Shader Write Acceleration Structure Read
Geometry Shader	Uniform Read Shader Read Shader Write Acceleration Structure Read
Fragment Shading Rate Attachment	Fragment Shading Rate Attachment Read
Early Fragment Tests	Depth Stencil Attachment Read Depth Stencil Attachment Write
Fragment Shader	Uniform Read Shader Read

Stage	Access Types
	Shader Write Input Attachment Read Acceleration Structure Read
Late Fragment Tests	Depth Stencil Attachment Read Depth Stencil Attachment Write
Color Attachment Output	Color Attachment Read Color Attachment Write

Compute pipeline:

Stage	Access Types
Draw Indirect	Indirect Command Read
Compute Shader	Uniform Read Shader Read Shader Write Acceleration Structure Read

Transfer pipeline:

Stage	Access Types
Transfer	Transfer Read Transfer Write

Host operations:

Stage	Access Types
Host	Host Read Host Write

Acceleration structure operations:

Stage	Access Types
Acceleration Structure Build	Draw Indirect Shader Read Transfer Read Transfer Write Acceleration Structure Read Acceleration Structure Write

Ray tracing pipeline:

Stage	Access Types
Draw Indirect	Indirect Command Read
Ray Tracing Shader	Uniform Read Shader Read Shader Write Acceleration Structure Read

Render loop

In this diagram, time progresses from top to bottom. The C++ implementation of this render loop can be found on my (WIP) Pumpkin game engine repository.

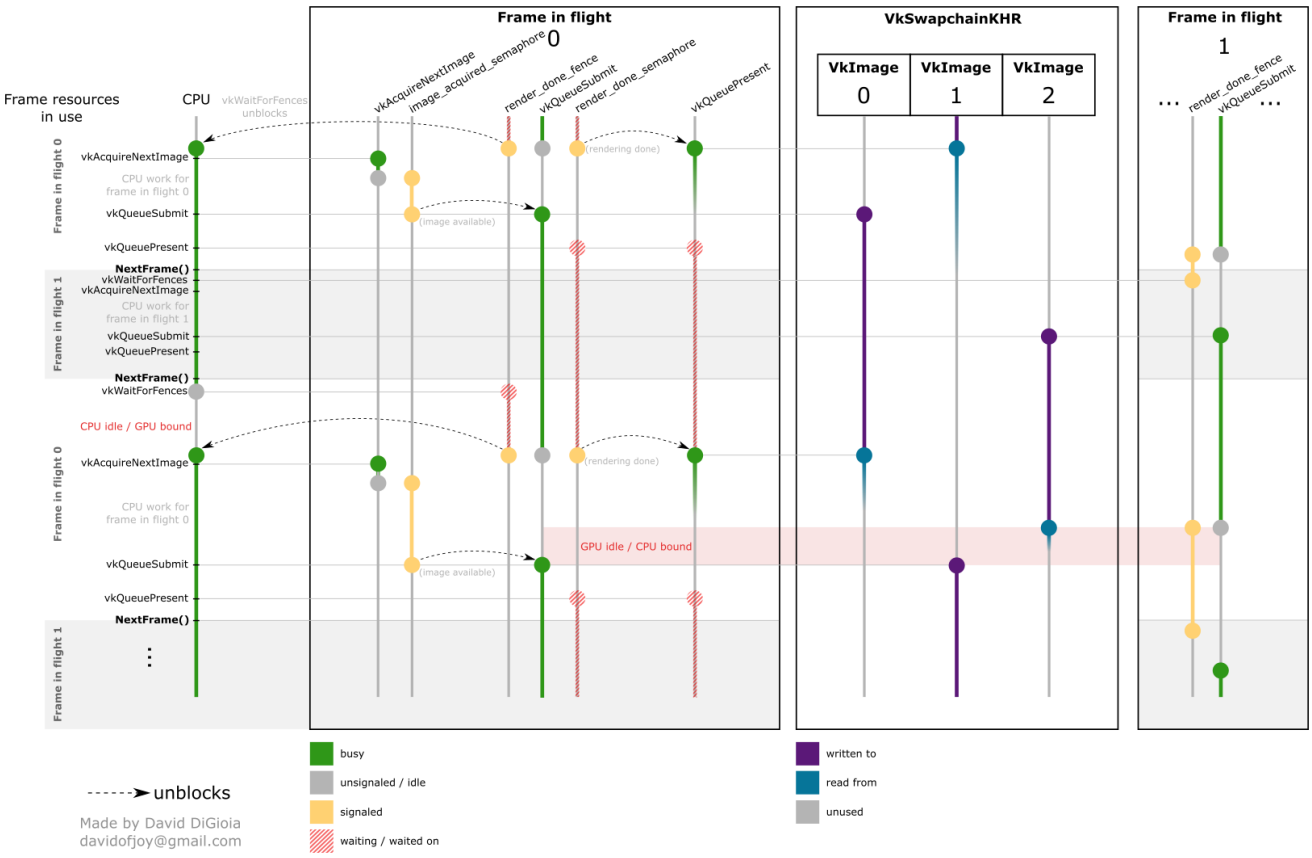
The sections of the CPU marked as "CPU work" refer specifically to writing to resources that are read by the GPU during rendering for that frame-in-flight. For example, updating descriptors. This is where I update all of my render objects' transforms by writing into their UBOs.

Notice that it is ambiguous when `vkQueuePresentKHR` is done presenting because the API does not accept any sync primitives to signal when it's done. Instead you can only know the swapchain image is done being used when `vkAcquireNextImageKHR` returns its index and signals a sync primitive indicating it's ready. Alternatively, you could use the VK_KHR_present_wait extension if you want to explicitly wait for presentation to finish.

Even though there are two different timelines drawn of `vkQueueSubmit` for each frame in flight, they both are submitting to the same queue.

Note: `vkAcquireNextImageKHR` won't signal the semaphore/fence until the image is ready, and the image won't be ready until enough previously acquired images are released with `vkQueuePresentKHR`. `vkAcquireNextImageKHR` will return the code `VK_NOT_READY` to indicate that the semaphore/fence hasn't been signaled immediately, but it will signal later once an image is acquired.

If you have vsync enabled, `vkQueuePresentKHR` is the function that will block until the next vsync cycle, at least on my GeForce GTX 1080.



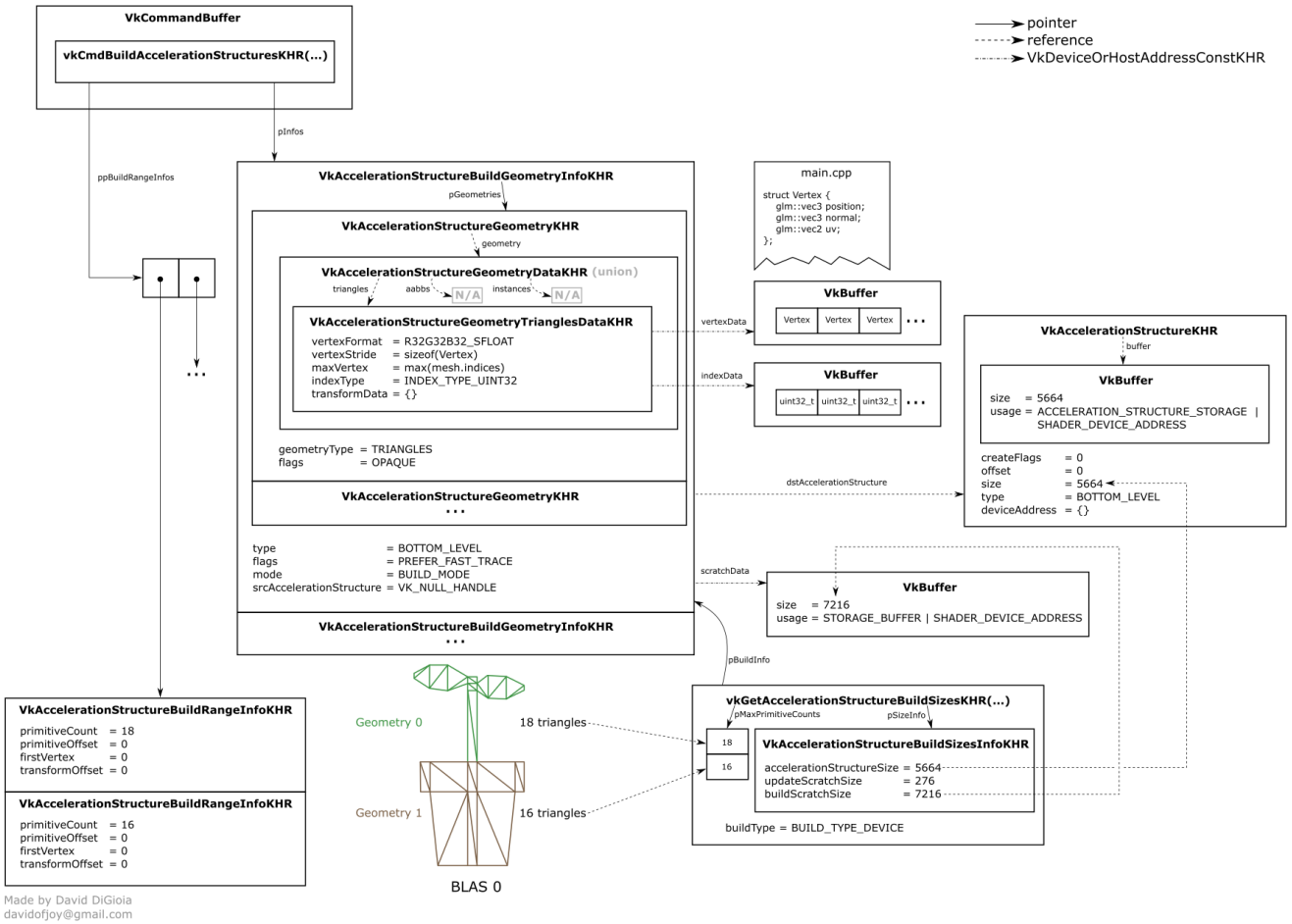
Ray tracing

Ray tracing in Vulkan consists of building acceleration structures, creating a ray tracing pipeline and shader binding table (SBT), and then tracing the rays with `vkCmdTraceRaysKHR(...)`. There is also ray VK_KHR_ray_query for casting rays in existing shaders and does not require a ray tracing pipeline, but that is not discussed here.

Most of the work of building the acceleration structures is done by the driver, but the application developer is responsible for placing instances within a top-level acceleration structure (TLAS),

grouping their primitives into bottom-level acceleration structures (BLASes) and within that BLAS grouping the primitives into geometries. How this is done can have a significant impact on performance. I've written an article on GPUOpen that goes into detail of best practices for ray tracing performance.

The first diagram shows the Vulkan objects needed to build a BLAS.

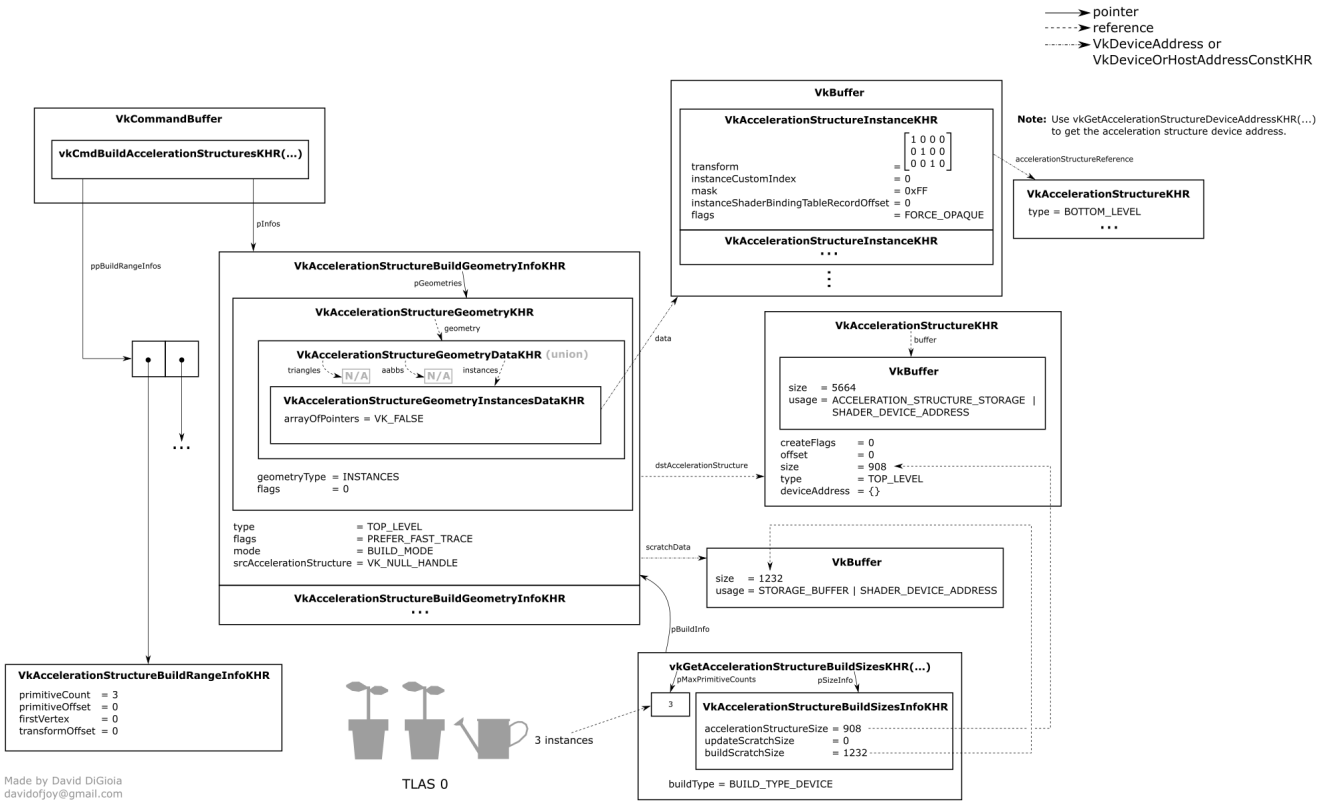


Note that almost no implementation supports `VkPhysicalDeviceAccelerationStructureFeaturesKHR::accelerationStructureHostCommands` so most likely you will need to build the acceleration structures on the device, as pictured in the diagrams. This makes compacting BLASes more complicated because it requires two queue submissions. The process to compact a BLAS is as follows:

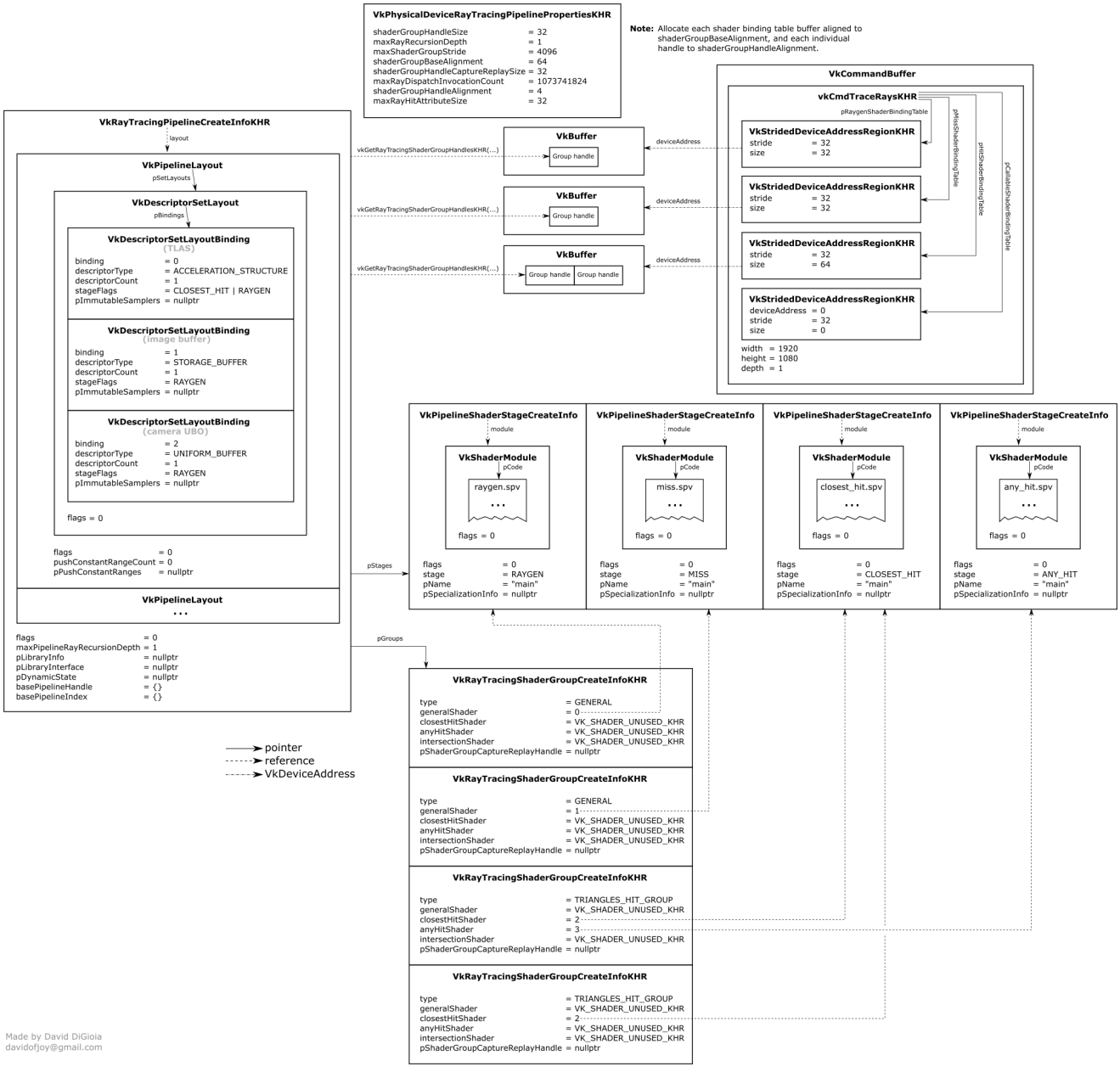
1. Add `VK_BUILD_ACCELERATION_STRUCTURE_ALLOW_COMPACTION_BIT_KHR` flag to `VkAccelerationStructureBuildGeometryInfoKHR::flags` for original acceleration structure that is built.
2. Create original acceleration structure.

3. Create `VkQueryPool` with a `VkQueryPoolCreateInfo::queryType` of `VK_QUERY_TYPE_ACCELERATION_STRUCTURE_COMPACTED_SIZE_KHR`.
4. Query the compacted size with `vkCmdWriteAccelerationStructuresPropertiesKHR(...)`.
5. Submit the command buffer then get the query results with `vkGetQueryPoolResults(...)`.
6. Create a `VkAccelerationStructureKHR` with a `VkBuffer` with compacted size from query.
7. Start recording new command buffer.
8. Copy the original acceleration structure to the compacted one using `vkCmdCopyAccelerationStructureKHR(...)` with `VkCopyAccelerationStructureInfoKHR::mode` set to `VK_COPY_ACCELERATION_STRUCTURE_MODE_COMPACT_KHR`.
9. Submit command buffer.

The next diagram shows the Vulkan objects needed to build a TLAS.



Lastly, the ray tracing pipeline and shader binding table.



If these diagrams have been helpful to you, consider supporting me:

